

05. Modelagem

1. Objetivo da modelagem

A modelagem do ExtraUFLA busca representar a estrutura estática, o comportamento dinâmico e as interações dos usuários com o sistema. Os diagramas foram produzidos a partir dos requisitos definidos na Sprint 2 (04_requisitos.md) e cobrem tanto funcionalidades já implementadas (RF01, RF02) quanto funcionalidades planejadas (RF04, RF05, RF11, etc.).

Todos os diagramas utilizam **MermaidJS**, permitindo versionamento no repositório e renderização direta no GitHub.

2. Modelo de domínio

O sistema ExtraUFLA é composto por oito entidades principais organizadas em torno de **usuários**, **organizações** e **processos seletivos**.

Entidades principais

- **User** — Usuário do sistema (aluno ou admin). Possui e-mail institucional @ufla.br (RN01) e role que define seu nível de acesso.
- **Organization** — Organização extracurricular (empresa júnior, projeto de extensão ou núcleo de estudo). Classificação obrigatória por tipo (RN04).
- **OrganizationAdmin** — Entidade associativa entre User e Organization. Suporta múltiplos administradores por organização (RN03).
- **SelectiveProcess** — Processo seletivo com status automático derivado das datas de início e fim (RN02).
- **Course** — Curso de graduação para personalização do conteúdo (RF03).
- **Session** — Sessão ativa do usuário, controlada pelo better-auth.
- **Account** — Credenciais de autenticação (senha hashada com bcrypt).
- **Verification** — Tokens de verificação de e-mail e recuperação de senha.

Enums

- **Role** — user | admin (define se o usuário é admin da plataforma — RF05)
 - **OrgType** — Empresa_Junior | Projeto_Extensao | Nucleo_Estudo (RN04)
 - **OrgStatus** — Ativa | Inativa
 - **ProcessStatus** — Aberto | Encerrado | Agendado (RN02)
-

3. Diagramas propostos

3.1 Diagrama de casos de uso

Objetivo: Representar os atores do sistema e as funcionalidades disponíveis para cada perfil de acesso.

Atores principais: - **Visitante** — navega no catálogo sem autenticação (RF16) - **Aluno** — autenticado, pode criar organizações e editar perfil - **Admin Organização** — gerencia dados da organização e processos seletivos - **Admin Plataforma** — gerencia toda a plataforma (RF05)

Casos de uso principais: - 13 casos de uso cobrindo RF01-RF16 - Relações <<include>> entre CU03↔CU04, CU03↔CU05, CU06↔CU07 - Relação <<extend>> entre CU02↔CU13 (recuperação de senha estende autenticação)

flowchart LR

```
subgraph ExtraUFLA["Sistema ExtraUFLA"]
    CU01["CU01 - Cadastrar-se"]
    CU02["CU02 - Autenticar-se"]
    CU03["CU03 - Navegar no catálogo"]
    CU04["CU04 - Filtrar organizações"]
    CU05["CU05 - Buscar organizações"]
    CU06["CU06 - Ver detalhes da organização"]
    CU07["CU07 - Ver processos seletivos"]
    CU08["CU08 - Cadastrar organização"]
    CU09["CU09 - Gerenciar admins da organização"]
    CU10["CU10 - Criar processo seletivo"]
    CU11["CU11 - Gerenciar plataforma"]
    CU12["CU12 - Editar perfil"]
    CU13["CU13 - Recuperar senha"]
end
```

end

Visitante(["Visitante"])

Aluno(["Aluno"])

AdminOrg(["Admin Organização"])

AdminPlat(["Admin Plataforma"])

Visitante --> CU01

Visitante --> CU02

Visitante --> CU03

Visitante --> CU13

Aluno --> CU03

Aluno --> CU06

Aluno --> CU08

Aluno --> CU12

AdminOrg --> CU09

AdminOrg --> CU10

AdminPlat --> CU11

CU03 --- CU04

CU03 --- CU05

CU06 --- CU07

3.2 Diagrama de classes

Objetivo: Representar a estrutura estática do sistema com entidades, atributos, métodos e relacionamentos.

Principais classes: - User, Session, Account, Verification (camada de autenticação — já implementada) - Organization, OrganizationAdmin, SelectiveProcess (camada de domínio — planejada) - Course (entidade auxiliar — planejada)

```
classDiagram
    direction TB

    class User {
        +id: string
        +name: string
        +email: string
        +emailVerified: boolean
        +image: string
        +role: Role
        +courseId: string
        +createdAt: Date
        +updatedAt: Date
    }

    class Session {
        +id: string
        +token: string
        +expiresAt: Date
        +ipAddress: string
        +userAgent: string
        +createdAt: Date
        +updatedAt: Date
    }

    class Account {
        +id: string
        +providerId: string
        +accountId: string
        +password: string
        +createdAt: Date
        +updatedAt: Date
    }

    class Organization {
        +id: string
        +name: string
        +type: OrgType
    }
```

```

        +description: string
        +area: string
        +contact: string
        +socialMedia: string
        +logo: string
        +status: OrgStatus
        +creatorId: string
        +createdAt: Date
        +updatedAt: Date
    }

class OrganizationAdmin {
    +id: string
    +addedAt: Date
}

class SelectiveProcess {
    +id: string
    +title: string
    +description: string
    +vacancies: int
    +startDate: Date
    +endDate: Date
    +requirements: string
    +status: ProcessStatus
    +createdAt: Date
    +updatedAt: Date
}

class Course {
    +id: string
    +name: string
    +code: string
    +department: string
}

class Role {
    <<enumeration>>
    user
    admin
}

class OrgType {
    <<enumeration>>
    Empresa_Junior
    Projeto_Extensao
    Nucleo_Estudo
}

```

```

class OrgStatus {
    <<enumeration>>
    Ativa
    Inativa
}

class ProcessStatus {
    <<enumeration>>
    Aberto
    Encerrado
    Agendado
}

User "1" --> "*" Session : possui
User "1" --> "*" Account : possui
User "*" --> "0..1" Course : matriculado em
User "1" --> "*" OrganizationAdmin : gerencia
Organization "*" --> "1" User : criado por
Organization "1" --> "*" OrganizationAdmin : possui
Organization "1" --> "*" SelectiveProcess : oferece
OrganizationAdmin "*" --> "1" Organization : administra
OrganizationAdmin "*" --> "1" User : referência
SelectiveProcess --> ProcessStatus : status
Organization --> OrgType : tipo
Organization --> OrgStatus : status
User --> Role : role

```

3.3 Diagramas de sequência

Objetivo: Representar a interação entre os componentes do sistema nos fluxos principais.

Processos representados: 1. **Cadastro e autenticação** — fluxo de registro com validação @ufla.br e login com sessão (RF01, RF02, RN01) 2. **Criação de organização e gerenciamento de admins** — fluxo de cadastro de org e adição de novos administradores (RF04, RN03)

Sequência 1: Cadastro e Autenticação

```

sequenceDiagram
    actor Aluno
    participant FE as Frontend
    participant BE as Backend
    participant DB as Database

    Note over Aluno,DB: Cadastro (RF01)

```

```

Aluno->>FE: Acessa página de cadastro
FE->>Aluno: Exibe formulário (nome, email, senha)
Aluno->>FE: Preenche e envia dados
FE->>BE: POST /api/auth/sign-up {name, email, password}

```

```

alt E-mail não é @ufla.br (RN01)
    BE->>FE: 400 "Utilize seu e-mail @ufla.br"
    FE->>Aluno: Exibe erro de validação
else Dados válidos
    BE->>DB: INSERT User
    DB->>BE: User created
    BE->>FE: 200 Success
    FE->>Aluno: Redireciona para login
end

```

Note over Aluno,DB: Autenticação (RF02)

```

Aluno->>FE: Insere credenciais no login
FE->>BE: POST /api/auth/sign-in {email, password}
BE->>DB: SELECT User WHERE email
DB->>BE: User record

```

```

alt Credenciais inválidas
    BE->>FE: 401 "Credenciais inválidas"
    FE->>Aluno: Exibe erro
else Credenciais válidas
    BE->>BE: Verifica senha (bcrypt)
    BE->>DB: INSERT Session
    DB->>BE: Session created
    BE->>FE: 200 {session token, user}
    FE->>FE: Armazena cookie de sessão
    FE->>Aluno: Redireciona para /dashboard
end

```

Sequência 2: Criação de Organização e Gerenciamento de Admins

sequenceDiagram

```

actor Aluno
participant FE as Frontend
participant BE as Backend
participant DB as Database

```

Note over Aluno,DB: Cadastro de Organização (RF04)

```

Aluno->>FE: Acessa "Criar Organização"
FE->>BE: GET /api/auth/session
BE->>FE: {session valid, user}
FE->>Aluno: Exibe formulário (nome, tipo, descrição, área, contato, logo)

```

```

Aluno->>FE: Preenche e envia dados
FE->>BE: POST /api/organizations {name, type, description, ...}

alt Nome já existe
    BE->>FE: 409 "Já existe organização com esse nome"
    FE->>Aluno: Exibe erro
else Dados válidos
    BE->>DB: INSERT Organization {creatorId: currentUser}
    DB->>BE: Organization created
    BE->>DB: INSERT OrganizationAdmin {userId: currentUser, orgId}
    DB->>BE: Admin assigned
    BE->>FE: 201 {organization}
    FE->>Aluno: Redireciona para página da organização
end

Note over Aluno,DB: Adicionar Admin (RN03)

Aluno->>FE: Busca usuário por nome ou e-mail
FE->>BE: GET /api/users?search=xxx
BE->>DB: SELECT Users WHERE name LIKE OR email LIKE
DB->>BE: Resultados
BE->>FE: Lista de usuários
FE->>Aluno: Exibe resultados

Aluno->>FE: Seleciona usuário e confirma
FE->>BE: POST /api/organizations/:id/admins {userId}
BE->>BE: Verifica se requisitante é admin (RN03)
BE->>DB: INSERT OrganizationAdmin
DB->>BE: Success
BE->>FE: 201
FE->>Aluno: Exibe confirmação

```

4. Decisões de modelagem

Decisão	Justificativa
MermaidJS para todos os diagramas	Versionável no repositório, renderiza no GitHub, sem ferramentas externas
OrganizationAdmin como entidade associativa	Suporta RN03: múltiplos administradores por organização com relacionamento N:M
Role (enum) no User	Mapeia para o plugin admin do better-auth (RF05), diferenciando usuário comum de admin da plataforma
ProcessStatus como enum automático	Suporta RN02: status derivado das datas de início/fim, não definido manualmente

Decisão	Justificativa
Course como entidade separada	Normaliza dados e suporta RF03 (seleção de curso), mesmo adiada para Sprint 4
Diagramas de sequência para cadastro + criação de org	Fluxos mais críticos e ariteturamente significativos, exercitam RF01, RF02, RF04 e RN01, RN03
flowchart LR para casos de uso	MermaidJS não possui tipo nativo de diagrama de casos de uso UML; flowchart com subgraph aproxima a notação

5. Exemplo de descrição textual

O sistema ExtraUFLA possui as entidades **User**, **Organization**, **Selective-Process** e **Course**. Um **User** pode criar várias **Organizations** (tornando-se automaticamente **OrganizationAdmin** de cada uma). Cada **Organization** oferece múltiplos **SelectiveProcesses** com status automático baseado nas datas (Aberto, Encerrado ou Agendado). **Users** podem pertencer a um **Course** e possuem um **Role** que define seu nível de acesso (usuário comum ou admin da plataforma). Dados do catálogo são públicos; dados de perfil são privados e visíveis apenas ao próprio aluno ou admins da plataforma (RN05).

6. Rastreabilidade requisitos-modelos

6.1 Requisitos Funcionais

Requisito	Caso(s) de Uso	Diagrama(s)	Classe(s)
RF01	CU01	Seq 3.3.1	User, Account
RF02	CU02	Seq 3.3.1	User, Session
RF03	CU12 (parcial)	Classes	User, Course
RF04	CU08	Seq 3.3.2	Organization, OrganizationAdmin
RF05	CU11	Classes	User (role=admin)
RF06	CU03	Casos de Uso	Organization
RF07	CU06	Casos de Uso	Organization
RF08	CU07	Casos de Uso	SelectiveProcess
RF09	CU04	Casos de Uso	Organization (OrgType)
RF10	CU05	Casos de Uso	Organization
RF11	CU10	Seq 3.3.2	SelectiveProcess
RF12	—	—	SelectiveProcess
RF13	CU12	Casos de Uso	User
RF14	—	Classes	User, SelectiveProcess
RF15	CU13	Seq 3.3.1	User, Verification

Requisito	Caso(s) de Uso	Diagrama(s)	Classe(s)
RF16	CU03, CU06	Casos de Uso	Organization

6.2 Requisitos Não Funcionais

Requisito	Categoria	Aplicável a
RNF01	Usabilidade	Todos os diagramas (responsividade)
RNF02	Desempenho	Seq 3.3.1, 3.3.2 (LCP < 3s)
RNF03	Segurança	Seq 3.3.1 (bcrypt, JWT, HTTPS)
RNF04	Acessibilidade	Casos de Uso (interação com atores)
RNF05	Compatibilidade	Transversal (todos os fluxos)
RNF06	Confiabilidade	Infraestrutura (CI/CD, deploy)
RNF07	Usabilidade	Seq 3.3.1, 3.3.2 (feedback visual)

6.3 Regras de Negócio

Regra	Diagrama(s)	Classe(s)	Descrição
RN01	Seq 3.3.1	User	Validação @ufla.br no cadastro
RN02	Classes	SelectiveProcess (ProcessStatus)	Status automático por datas
RN03	Seq 3.3.2, Classes	OrganizationAdmin	Múltiplos admins por organização
RN04	Classes	Organization (OrgType)	Classificação obrigatória por tipo
RN05	Casos de Uso, Classes	User (role), OrganizationAdmin	Visibilidade por papel de acesso